

Grok Bot Team

Building an 24/7 Multi-Agent System: The SpaceXAI Playbook

Based on Grok Bot documentation and public SpaceXAI and Cursor engineering materials,
independently reworked and adapted into document form by @0xCodila

Independently compiled, August 2026 - not affiliated with SpaceXAI, xAI, SpaceX, or Cursor - and not endorsed

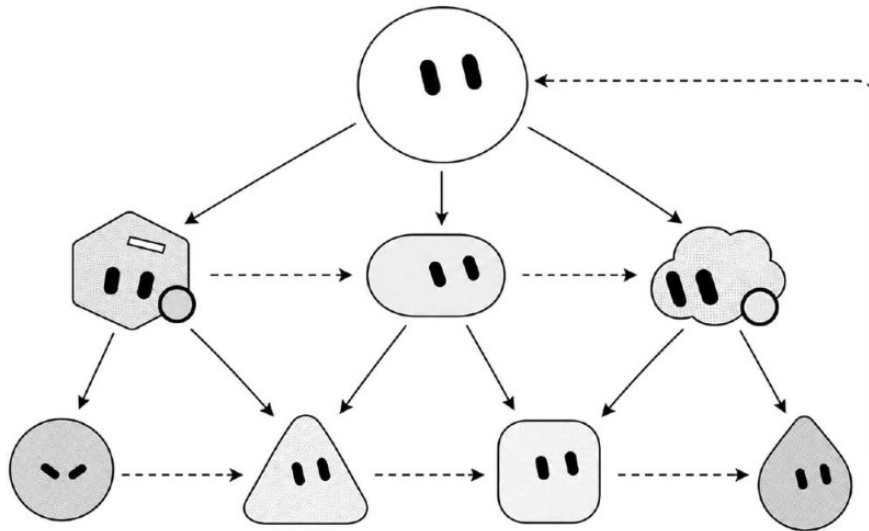


Fig. 1. Hierarchical Grok Bot team architecture. A Manager Bot delegates work to specialists, while typed handoffs, coordination paths, and a feedback loop keep the team moving from intake to verified delivery.

Abstract - Grok Bot can operate as more than a set of independent assistants. When Bots are assigned explicit ownership, reusable Skills, event-driven Routines, typed handoffs, verification rules, and approval boundaries, they become a persistent multi-agent system. This note presents a practical architecture for building that system from one repeatable workflow. A Manager Bot receives work, selects the next specialist, tracks incomplete stages, and returns to the user only when judgment is required. Specialist Bots work inside their own tools and cloud computers, preserve artifacts, pass structured results to the next owner, and continue while the user's laptop is closed. The central engineering problem is not how many Bots to create. It is how to make ownership, state, evidence, retries, and completion observable. We map the complete path from a single Bot to an always-on team that can execute, verify, and deliver recurring work with minimal human routing.

Index Terms - Grok Bot, multi-agent systems, agent orchestration, Skills, Routines, event triggers, structured handoffs, verification, approval boundaries.

I. INTRODUCTION

A chatbot produces responses. An agent performs actions. An agent system owns a workflow across time. Most first implementations create several named Bots and assign each one a professional title. This creates the appearance of a team, but not the operating structure of one. If every Bot receives a loose prompt and reports directly to the user, the human remains the router, memory layer, reviewer, and recovery system.

The correct starting point is one recurring workflow with a measurable finish. Define where work enters, which Bot owns every stage, what artifact must persist, what evidence proves completion, which failures can be retried, and which action requires approval.

A separate Bot should be introduced only when a stage requires a distinct objective, toolset, context, or decision boundary. The successful method is then saved as a Skill. A Routine or event trigger starts it automatically. The resulting system does not wait for the user to manually copy context from one conversation to the next.

Reliable orchestration depends on explicit contracts. Each handoff should state the task, input, expected artifact, evidence, deadline, next owner, and escalation condition. The Manager coordinates those artifacts rather than repeatedly summarizing entire chats.

A. Who This Note Is For

This note is for operators, founders, engineers, analysts, and small teams who already have a repeatable digital workflow and want Grok Bots to own it end to end. No custom agent framework is required. The reader should be able to describe the current manual process, identify the applications it touches, and define what a finished result looks like. The architecture is useful for research, inbox operations, reporting, customer follow-up, content production, and other workflows that can be observed and verified. It is not intended for irreversible actions without a human approval boundary.

B. Contributions

This synthesis makes four practical contributions. First, it maps Grok Bot capabilities to a complete system for intake, routing, execution, verification, approval, and delivery. Second, it defines a staged build path from one measured Bot to parallel specialists and a Manager Bot. Third, it provides copy-ready role definitions, task contracts, Skills, Routines, handoff schemas, acceptance tests, retry rules, and human checkpoints. Fourth, it separates reversible automation from actions that require approval. The goal is a system that remains understandable as it becomes more autonomous. Each stage is tested before the next is added.

II. SYSTEM MODEL AND MATURITY LADDER

From a useful Bot to an observable operating system

TABLE I. GROK BOT SYSTEM MATURITY

Level	Mode	Operating Model	Exit Test
0	Chat	User routes every task	Useful answer
1	Role	One durable Bot owns a job	Consistent output
2	Skill	Method is saved and reusable	Repeatable quality
3	Routine	A trigger starts the work	Unattended run
4	Team	Manager routes specialists	Parallel delivery
5	Governed	Evidence, limits, recovery	Reliable system

A. The Unit of Design Is the Workflow

A Bot is a worker. A system is the complete path by which work enters, receives an owner, produces an artifact, is verified, and reaches a finish state. Naming five Bots does not create a team if the user still copies context, chooses every next step, and checks every result. The first design artifact should therefore be a workflow map, not a roster.

B. Six Invariants

Every production workflow needs six invariants: one current owner, explicit state, a durable artifact, observable evidence, a bounded retry policy, and a clear approval boundary. If any invariant is missing, the human quietly becomes the memory layer or recovery system. The architecture on page 1 expresses these invariants as a graph of ownership rather than as a collection of chats.

C. Minimum Observable State

- **task_id** - stable identity across every Bot and application.
- **owner** - exactly one Bot accountable for the next action.
- **status** - queued, running, waiting, verifying, done, or escalated.
- **artifact** - file, record, message draft, report, test log, or video.
- **evidence** - what proves that the artifact satisfies the request.
- **next_deadline** - when the Manager checks progress or escalates.

Pseudocode 1 - The Smallest Complete Loop

```
while workflow.open:
    event = observe_source()
    owner = manager.route(event)
    artifact = owner.execute(event)
    verdict = verifier.check(artifact)
    if verdict.pass: manager.advance()
    elif verdict.retryable: owner.retry()
    else: manager.escalate_to_human()
```

D. The Real Scaling Metric

Do not count Bots, prompts, or messages. Count completed workflow instances that required no manual routing and still produced acceptable evidence. A system is improving when completion rises while rework, unnecessary approvals, cost per completed task, and recovery time fall. This metric prevents a visually impressive team from hiding a large human coordination tax.

E. Author's Synthesis

Grok Bot provides persistent roles, tools, cloud computers, memory, collaboration, and Routines [1]. Cursor engineering materials add verification, strict environments, and cloud execution [5]-[10]. The playbook combines them into one operating model. It is not an internal SpaceXAI specification and should be adapted to the reader's own risk profile.

"A team of always-on agents" is the workshop's simplest description of Grok Bot.
- Amrita Venkatraman [1]

F. Define a Service Level Objective

Treat the workflow as a small service. State how quickly new work must be acknowledged, when an owner must be assigned, how long execution may remain silent, and what evidence is required before completion. A useful initial objective is not perfect autonomy. It is predictable movement: every task is either progressing, waiting on a named dependency, ready for verification, or escalated with a reason. This turns an impressive demonstration into an operating system that can be inspected and improved.

Operational Review Questions

- Can the Manager name the current owner without rereading the conversation?
- Can a human see the latest artifact and the evidence attached to it?
- Can a failed task resume without repeating every completed action?
- Can the system stop safely when a tool, credential, or instruction becomes uncertain?

G. State Before Scale

Do not add another specialist until one complete workflow exposes its state clearly. More workers multiply ambiguity when ownership and evidence are weak. Once the state model is reliable, additional Bots become replaceable execution capacity rather than new sources of hidden coordination work.

III. CHOOSING THE FIRST WORKFLOW

Automate one measured loop before building a team

TABLE II. CANDIDATE WORKFLOW SCORECARD

Candidate	Repeat	Observe	Reverse	Tool Scope	Start?
Daily inbox briefing	5	5	5	4	Yes
Research digest	4	4	5	4	Yes
Expense filing	4	4	2	3	Later
External outreach	4	3	2	3	Approval
One-off strategy	1	2	5	5	No

A. Start With a Repeated Source of Work

The strongest first workflow appears at least weekly, enters through a stable source, touches a bounded set of tools, and ends in an artifact that can be checked. Good candidates include a morning briefing, research digest, customer issue triage, release-note draft, or scheduled data-quality report. Avoid the instruction 'organize my entire life.' It has no stable finish condition and no useful baseline.

B. The Five-Question Filter

- Does the same work recur with similar inputs?
- Can a stranger identify when the task is complete?
- Can a failed action be reversed without material damage?
- Can the required accounts and tools be narrowly scoped?
- Does the workflow consume enough time to justify setup and supervision?

C. Write the Finish Line First

Define the deliverable before defining the Bot. 'Review the inbox' is activity. 'Post one Slack briefing by 08:00 with every newsletter linked, three decisions highlighted, and no email sent' is a finish line. The more concrete the artifact and acceptance criteria, the less the Bot must guess and the easier it becomes to verify the run.

Template 1 - Workflow Canvas

```
NAME: Morning Briefing
SOURCE: Gmail label /newsletters
TRIGGER: weekdays at 07:30
OWNER: Inbox Manager
OUTPUT: one Slack briefing
PASS: links present; duplicates removed
APPROVAL: none for draft; ask before reply
TIMEOUT: 20 min; RETRIES: 1
```

D. Establish the Manual Baseline

Run the workflow manually three times. Record elapsed time, applications opened, decisions made, common exceptions, and the evidence you used to judge quality. This produces both a realistic ROI baseline and the raw material for the first Skill. Without a baseline, a faster Bot can appear successful even when it omits the work that mattered.

TABLE III. BASELINE LOG

Run	Minutes	Exceptions	Quality 1-5	Rework
1	—	—	—	—
2	—	—	—	—
3	—	—	—	—

E. Exit Test for Page 3

Proceed only when one workflow has a named source, one owner, one artifact, measurable acceptance criteria, a permission rule, a timeout, and a baseline. The first system should feel almost too small. Reliability is learned more cheaply on a bounded loop than across a network of twenty Bots.

F. Score Candidate Workflows

Rank candidate workflows on five dimensions from one to five: repetition, input stability, reversibility, evidence quality, and economic value. Subtract a risk score for sensitive data, irreversible writes, unclear ownership, and dependence on undocumented judgment. The best first workflow is usually frequent, boring, measurable, and easy to undo. A rare executive decision may look valuable, but it produces too few observations to improve the system safely.

Reject the First Workflow When

- the finish line is a feeling rather than an observable artifact;
- the source changes faster than the Skill can be maintained;
- failure would create an external commitment that cannot be reversed;
- the human cannot describe the current manual method consistently.

G. Instrument the Manual Run

Perform the workflow manually three times while recording inputs, decisions, tool actions, elapsed time, failure points, and final evidence. This trace becomes the first Skill specification and the baseline for measuring improvement. Without a baseline, a faster agent may simply be skipping steps that were previously invisible.

IV. THE MANAGER BOT CONTRACT

One router, one ledger, one point of escalation

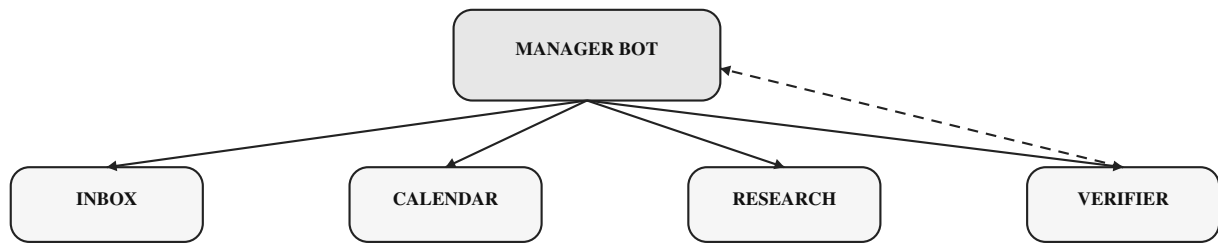


Fig. 2. The Manager owns routing and state; specialists own artifacts; the Verifier returns evidence.

A. What the Manager Owns

The Manager Bot receives the request, normalizes it into a task record, chooses the specialist, tracks dependencies, requests evidence, and decides whether to advance, retry, or escalate. It owns the workflow state, not every piece of specialist work. This separation prevents the Manager's context from filling with low-level execution details.

B. What the Manager Must Not Do

A Manager should not browse every site, rewrite every artifact, or become the universal backup specialist. If it repeatedly performs a specialist's job, the role boundary is wrong. The Manager should consume compact handoff records and artifact links, then route the next action. It returns to the human only for an approval, ambiguity, policy exception, or exhausted retry budget.

Template 2 - Manager Charter

```

ROLE: Workflow Manager
GOAL: deliver one verified final artifact
INPUTS: task, priority, deadline, policy
ROUTE BY: expertise, tool, risk, capacity
TRACK: owner, status, artifact, evidence
NEVER: invent completion or hide failure
ESCALATE: ambiguity, blocked access,
          failed verification, high risk
  
```

C. Deterministic Routing Before Model Judgment

Use explicit routing rules for common cases and reserve model judgment for genuine ambiguity. A calendar event belongs to the Calendar Bot. A repository change belongs to the engineering team. A request to send money or publish externally belongs behind approval. Deterministic routes are cheaper, auditable, and easier to test than open-ended selection.

Pseudocode 2 - Bounded Router

```

if task.risk >= HIGH: return HUMAN
if task.tool == 'calendar': return CALENDAR
if task.tool == 'gmail': return INBOX
if task.kind == 'research': return RESEARCH
candidate = classify(task, role_catalog)
return candidate if candidate.conf > .85
else return HUMAN
  
```

D. Manager Health Checks

- No task is simultaneously owned by two specialists.
- Every waiting state names the dependency and a check time.
- Every completed state points to evidence, not only a claim.
- Every retry increments an attempt counter and preserves the prior artifact.
- Every human interruption explains the decision required in one sentence.

E. One Inbox for the Human

The workshop demonstrates a Chief of Staff that delegates to utility Bots and tags the relevant specialists [1]. Preserve that advantage by using one human-facing thread. Specialists may coordinate in their own threads, but only the Manager should deliver the daily summary, approval request, or final result. This reduces interruption without hiding traceability.

F. The Manager Ledger

The Manager should maintain a compact ledger rather than a long narrative. Each row contains task identity, priority, current owner, state, artifact link, evidence link, next deadline, retry count, and approval status. Specialists update the record at handoff boundaries. The Manager reads the ledger first and conversations only when a field is ambiguous. This preserves continuity when work spans hours, devices, or multiple Bots.

A Complete Escalation Packet

- the decision required from the human;
- the recommended option and the reason for it;
- the alternatives already tested;
- the cost of waiting and the safest default action.

G. Manager Acceptance Test

Give the system five mixed tasks, remove one specialist, delay one input, and create one policy exception. A good Manager reroutes available work, marks blocked work explicitly, preserves completed artifacts, and asks the human one compact question. It does not silently absorb every specialist role or restart the entire workflow.

V. SPECIALIST ROLE DESIGN

Split work by objective, tools, and decision boundary

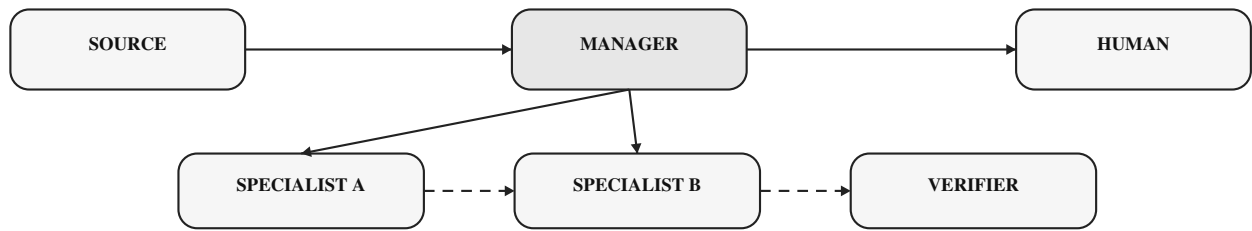


Fig. 3. Create specialists for durable boundaries, not for every conversational step.

A. A Persona Is a Durable Operating Boundary

The Grok Bot workshop describes a move from a task-based model to a persona-based model [1]. The useful part of a persona is not a playful name. It is durable ownership: a stable objective, preferred tools, reusable Skills, memory, acceptance tests, and a known escalation path. A role becomes valuable when repeated tasks benefit from the same accumulated context.

Template 3 - Specialist Role Card

NAME: Research Analyst
 OWNS: evidence collection and synthesis
 TOOLS: browser, Drive, approved databases
 INPUT: question + source requirements
 OUTPUT: brief + source ledger
 PASS: every claim maps to a source
 NEVER: publish or contact a source
 ESCALATE: conflicting primary evidence

B. When to Create Another Bot

Create a specialist only when a stage requires a distinct objective, tool environment, permission set, long-lived context, or verification method. Do not split a workflow merely because a task has several steps. A single Bot with one strong Skill is usually better than three Bots whose only job is to pass prose to one another.

TABLE IV. ROLE SEPARATION TEST

Signal	Keep One Bot	Create Specialist
Tools	Same apps	Different accounts or environment
Context	Short shared task	Durable domain memory
Risk	Same permission	Different approval boundary
Quality	Same verifier	Distinct evidence method
Load	Sequential	Independent parallel work

C. Useful Starter Roles

- **Inbox Manager** - triage, summarize, draft, and surface decisions.
- **Calendar Coordinator** - propose slots and detect conflicts.
- **Research Analyst** - collect evidence and maintain a source ledger.
- **QA Verifier** - test outputs and attach proof before completion.
- **On-Call Monitor** - watch an event source and escalate real incidents.
- **Publisher** - transform approved artifacts into final distribution formats.

D. Avoid All-to-All Teams

The workshop shows Bots talking to each other, yet also notes that only relevant Bots need to be pulled into a task [1]. Encode that discipline. Specialists should communicate through explicit dependencies or the Manager. All-to-all chat increases duplicated work, conflicting decisions, context cost, and the chance that nobody is clearly accountable.

E. Capacity Is Not a Design Goal

A participant reported using roughly 12-15 Bots, but the presenter declined to prescribe a universal limit [1]. Treat that as an existence proof, not a target. Begin with one Manager and two or three specialists. Add a Bot only after the existing team completes the workflow reliably and a measurable bottleneck remains.

F. Role Economics

A specialist deserves its own durable identity when it has a stable objective, recurring tool set, distinct permissions, and an evaluation method that differs from other work. If two proposed Bots share the same tools, evidence, and approval boundary, keep one Bot and express the variation as a Skill. Separate identities create maintenance cost: more prompts, credentials, memory, monitoring, and failure modes.

Role Boundary Test

- What artifact does this Bot uniquely own?
- Which permissions can be removed from every other Bot?
- Which metric reveals whether this role is improving?
- What event transfers ownership into and out of the role?

G. Prevent Duplicate Agency

Never assign two Bots the same open-ended objective without defining how their outputs converge. Duplication is useful only when it is intentional, such as independent research, adversarial review, or alternative implementation. The Manager must name the selector or verifier that decides what survives. Otherwise parallelism produces a larger pile of unresolved choices for the human.

VI. TOOLS, COMPUTERS, AND ACCESS

Give each Bot the smallest environment that can finish its job

TABLE V. TOOL SELECTION ORDER

Layer	Use When	Strength	Primary Risk
Plugin / API	Stable structured operation	Fast and auditable	Excess scope
MCP	Custom internal tool	Composable interface	Server trust
Computer use	No useful API exists	Works across real UI	UI drift
Human	Irreversible judgment	Accountable decision	Latency

A. Dedicated Cloud Computers

Grok Bots can work inside applications and websites from their own computers, and the public workshop describes them continuing while the user's laptop is closed [1]. Cursor's cloud-agent documentation similarly describes isolated virtual machines with desktop and browser control [5]. The practical implication is that the workflow must be designed as a remote operating environment, not as a long chat session.

B. Prefer Structured Interfaces

Use a plugin, API, or MCP when it exposes the required operation reliably. Use computer control when the workflow crosses an application without a suitable interface or when the Bot must verify a visual result. A browser can reach more surfaces, but selectors, layouts, and authentication flows change. The safest design selects the narrowest tool that can still produce the required evidence.

C. Access Setup Checklist

- Use a dedicated account or role where the application supports one.
- Grant only the folders, labels, projects, or records needed by the workflow.
- Keep financial, deletion, publishing, and external-send actions behind approval.
- Test authentication expiry and recovery before enabling a Routine.
- Log the account, environment, and tool used for every material action.
- Remove unused integrations and review permissions on a fixed schedule.

G. Prefer the Narrowest Reliable Surface

Use a structured API or MCP tool when it exposes the required operation and returns reliable evidence. Use the computer when the workflow depends on a visual interface, authentication flow, unsupported portal, or human-like navigation. The correct surface is the one that minimizes ambiguity while preserving the proof needed for verification.

Template 4 - Tool Boundary

```
ALLOW read: inbox/newsletters
ALLOW write: slack/#daily-briefing
ALLOW draft: email replies
ASK before: send_email, shared_calendar
DENY: delete_mail, payment, admin_change
NOTIFY: external write or repeated failure
```

D. Teach Through Demonstration

The workshop demonstrates a user performing a task on the Bot's computer and saving the observed workflow as a Skill [1]. Use demonstration for navigation, field selection, and preference capture, then edit the generated instructions. A recorded path is an initial trace, not a complete specification. Add examples, anti-patterns, timeouts, and acceptance tests before unattended use.

E. Remote Does Not Mean Unbounded

Cursor notes that autonomous cloud execution changes the threat model because agents can auto-run actions and may encounter prompt injection [6]. A dedicated computer improves continuity, but it does not remove the need for narrow credentials, network controls, approval rules, and evidence. Autonomy should expand only after the environment is easier to inspect and constrain than the manual process it replaces.

F. Session and Credential Design

A cloud computer makes browser workflows possible, but it also creates durable sessions that must be governed. Use a dedicated account where possible, separate read and write capabilities, and record which Bot may access each application. Credentials should be revocable without rebuilding the Bot. When a site requests an unexpected permission, the workflow should pause and surface the exact screen instead of improvising.

Evidence for Computer Use

- capture the page or record identifier before and after a write;
- store screenshots for visual actions and logs for structured actions;
- record the account, application, timestamp, and intended effect;
- verify the external state rather than trusting a success message.

VII. SKILLS AS OPERATING PROCEDURES

Turn one successful run into a versioned method

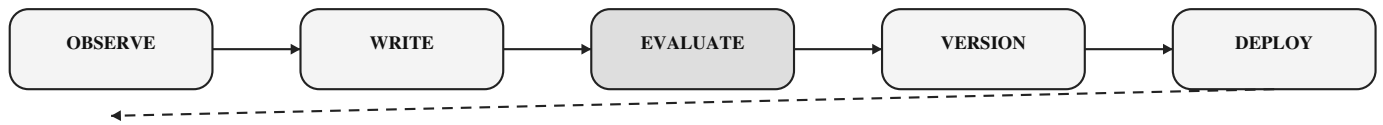


Fig. 4. Skill authoring is a closed loop: observe failures, encode the method, evaluate, version, and deploy.

A. The Purpose of a Skill

A Skill is a durable instruction package for a recurring method. The Grok Bot workshop describes Skills with a name, description, and instructions, and shows a demonstrated workflow becoming a reusable Skill [1]. Cursor also documents Skills as files that extend agents with domain-specific procedures [3]. The important shift is from correcting one run to improving every future run.

Template 5 - Skill Specification

```

SKILL: newsletter-briefing/v1
WHEN: inbox item matches approved sources
STEPS:
  1. extract title, author, link
  2. summarize only supported claims
  3. deduplicate repeated stories
  4. rank by decision relevance
OUTPUT: markdown briefing
PASS: every item has a working link
ANTI-PATTERN: do not draft replies
  
```

B. What High-Quality Skills Contain

- A narrow activation condition so the Skill is not invoked everywhere.
- Ordered steps with tool choices and stop conditions.
- Positive examples that demonstrate the desired artifact.
- Anti-patterns that show common but unacceptable shortcuts.
- Acceptance tests the Bot can execute without subjective guessing.
- A version number, owner, and date of the last verified run.

C. Convert Corrections Into Infrastructure

When a Bot fails, avoid adding a longer conversational reminder. Ask whether the failure belongs in the Skill, the environment, the verifier, or a hard policy. Lauren Tan describes incrementally building PStack by observing agent failure modes and encoding durable responses [9]. The result is an accumulating operating system rather than a growing pile of prompt history.

Pseudocode 3 - Skill Improvement

```

failure = inspect(run.trace)
if failure.method: patch(skill)
elif failure.environment: simplify(path)
elif failure.quality: strengthen(verifier)
elif failure.risk: tighten(policy)
run(eval_set)
publish(new_version) only if score rises
  
```

D. Evaluate Before Sharing

A shareable Skill can spread a good method or a hidden assumption. Test it on typical cases, edge cases, outdated interfaces, missing access, and deliberately ambiguous inputs. Lauren compares evals to unit tests for agents and recommends testing Skills across representative tasks and models [9]. Keep the prior version available so a regression can be reversed quickly.

E. Skill Versus Memory

Memory records what happened. A Skill defines what should happen. Do not rely on a Bot remembering one correction from an old conversation when the rule matters to every future run. Promote stable preferences, policies, and procedures into explicit Skills. Leave temporary facts, task-specific discussion, and exploratory reasoning in memory or the task ledger.

F. Version Skills Like Software

Every shared Skill should have an owner, version, change note, evaluation set, and rollback path. A correction learned from one task must not silently change unrelated workflows. Promote a new version only after it passes representative cases, known edge cases, and at least one deliberately malformed input. Keep the prior version available until the new one has survived real runs.

Minimum Regression Set

- one normal case with a known correct artifact;
- one incomplete input that must trigger clarification;
- one tool failure that must preserve progress;
- one policy boundary that must request approval.

G. Keep Instructions Executable

A Skill should contain observable actions and acceptance criteria, not motivational language. Replace 'do thorough research' with named sources, stopping rules, citation requirements, and a verifier. Replace 'write a great report' with a schema, audience, examples, prohibited claims, and an evidence checklist. The clearer the executable method, the less the Bot must guess.

VIII. ROUTINES AND EVENT TRIGGERS

A workflow becomes 24/7 only when work can start without a new prompt

TABLE VI. TRIGGER PATTERNS

Trigger	Example	Required Guard
Schedule	07:30 weekday briefing	Idempotency by date
Message	Tagged request in Slack	Approved channel
Event	New support ticket	Deduplicate event ID
Monitor	API error threshold	Rate and severity limit
Dependency	Artifact becomes ready	Schema validation

A. Routine Anatomy

A Routine combines a trigger, instructions, tools, output destination, approval policy, and retry behavior. The Grok Bot workshop demonstrates scheduled work and event-driven monitoring, while Cursor Automations describes schedule, message, and source-control triggers [1], [4]. A Routine is not merely a repeated prompt. It is a small production job with state and a finish test.

Template 6 - Routine Contract

```
ROUTINE: weekday-morning-briefing
TRIGGER: cron 30 7 * * 1-5
INPUT: unread items since last_success
SKILL: newsletter-briefing/v1
OUTPUT: Slack #daily-briefing
IDEMPOTENCY: briefing:{local_date}
RETRY: once after 10 min
APPROVAL: ask before any email send
```

B. Idempotency Prevents Duplicate Work

Every Routine needs a stable key that identifies one intended run. If the Bot restarts, a webhook is delivered twice, or a network call times out after succeeding, the same key should prevent duplicate calendar events, repeated messages, or multiple records. Store the last successful checkpoint and distinguish 'not attempted' from 'attempted but outcome unknown.'

C. Design for Late and Missing Inputs

A scheduled Routine should state what happens when no new work exists, an account is logged out, the source is delayed, or the expected dependency never appears. Silence is not success. Emit a small heartbeat for important routines and escalate only after a defined threshold. This keeps the human informed without turning every normal empty run into an interruption.

Pseudocode 4 - Safe Scheduled Run

```
key = routine.name + local_date
if ledger.completed(key): return NOOP
items = source.after(last_success)
if not items: record_heartbeat(key)
else:
    artifact = bot.run(skill, items)
    if verify(artifact): publish_once(key)
    else: retry_or_escalate(key)
```

D. Activation Checklist

- Run once manually with the exact production account and output destination.
- Run once in dry-run mode with external writes disabled.
- Simulate an empty input, expired login, duplicate event, and failed verifier.
- Confirm the stop switch and notification path from another device.
- Activate at low frequency, inspect three successful runs, then increase cadence.

E. 24/7 Is a Continuity Property

A cloud Bot can continue while the laptop is closed [1], [8], but a reliable 24/7 system also survives expired sessions, duplicated triggers, changed interfaces, missing dependencies, and partial failure. Availability is not measured by whether the Bot stayed online. It is measured by whether the workflow preserved state and recovered without corrupting the result.

F. Trigger Hygiene

A Routine needs more than a schedule. Define the event source, debounce window, idempotency key, quiet hours, expiration rule, and behavior when several events arrive together. A message trigger should not create five tasks because a thread produced five notifications. A scheduled Routine should not replay old work after a long outage unless replay is explicitly safe.

Routine Control Surface

- pause, resume, and run-now controls;
- last successful run and next expected run;
- current owner, retry count, and waiting reason;
- dead-letter queue for inputs that exhausted their budget.

G. Closed-Laptop Test

Start the Routine, close the local device, interrupt one dependency, and inspect the result later from another surface. The run passes only if state, artifacts, evidence, and pending approvals remain available. The phrase 'works while the laptop is closed' describes cloud continuity, not permission to operate without boundaries or review.

IX. TYPED HANDOFFS AND SHARED STATE

Pass artifacts and contracts, not entire conversations

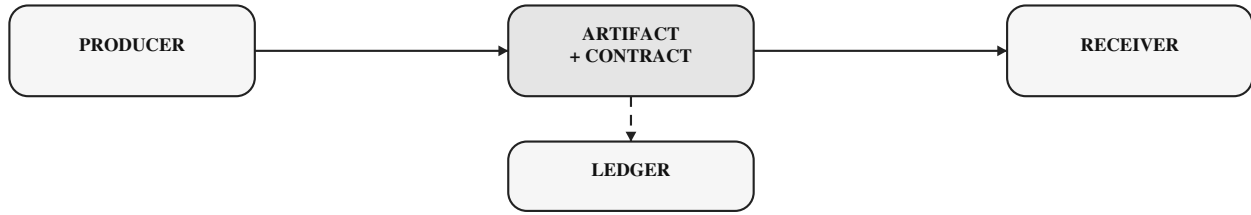


Fig. 5. A typed handoff transfers ownership through an artifact and a compact ledger record.

A. The Handoff Is an Interface

In the engineering demonstration, a backend Bot posts the API contract so the frontend Bot can begin before implementation is fully complete, and the QA Bot waits for both artifacts [1]. This is the correct mental model: a handoff is a typed interface between owners. It states what is ready, where it lives, how it can be checked, and what remains unresolved.

Template 7 - Handoff Record

```

task_id: lodging-042
from: Backend Bobby
to: Frontend Faye
artifact: api/openapi-lodging-v1.json
contract: GET /lodging?destination=
evidence: contract-test-184.log
assumptions: owner-only access
open_risks: provider rate limits
next_deadline: 2026-08-23T14:00Z
  
```

B. Required Fields

- Stable task and artifact identifiers.
- Current owner and explicitly named next owner.
- Schema, file, record, or link that the receiver can consume.
- Evidence already produced and the next acceptance test.
- Assumptions, unresolved risks, deadline, and escalation condition.

C. Shared State Is Smaller Than Shared Memory

The team does not need every Bot to reread every conversation. Maintain a compact ledger containing task state, artifact pointers, decisions, and evidence. Specialists can keep rich local context, while the Manager and downstream owners receive only the state needed to act. This lowers cost and reduces contradictory interpretations of an old chat.

TABLE VII. CHAT VERSUS CONTRACT

Question	Chat Summary	Typed Handoff
What is ready?	Implied	Named artifact
Who owns next?	Often unclear	Single owner
How to verify?	Narrative claim	Evidence pointer
Can it be retried?	Manual	Recorded state
Can it be audited?	Expensive	Structured ledger

Pseudocode 5 - Handoff Gate

```

record = parse_handoff(message)
require(record.task_id and record.artifact)
require(record.next_owner)
require(schema_valid(record.artifact))
require(evidence_exists(record.evidence))
ledger.append(record)
notify(record.next_owner)
  
```

D. Reject Ambiguous Completion

Statements such as 'done,' 'looks good,' or 'I handled it' should not advance a production workflow. The receiver should be able to open the artifact and independently run the named check. If the producer cannot point to evidence, the task remains in verifying or blocked state. This rule alone eliminates many false-positive completions.

E. Schema Evolution

Treat handoff records as versioned interfaces. Additive fields are usually safe; renamed meanings and removed fields are not. The receiver should reject a handoff whose schema version it does not understand, while the Manager preserves the producer's artifact for recovery. This is less glamorous than free-form collaboration, but it prevents one Bot's prompt update from silently breaking the entire team.

Handoff Quality Test

- a receiver can act without reading the producer's full transcript;
- the artifact has a stable location and explicit version;
- acceptance criteria are machine-checkable where possible;
- rejection returns a reason and the field that must change.

F. Separate Fact, Decision, and Evidence

Do not collapse all state into one summary paragraph. Facts describe the world, decisions record a chosen path, artifacts contain the work, and evidence proves acceptance. Keeping these objects separate makes it possible to replace a conclusion without rewriting history and to audit why the Manager routed the next step.

X. PARALLELISM WITHOUT COORDINATION DEBT

Fan out independent work, then converge through one verifier

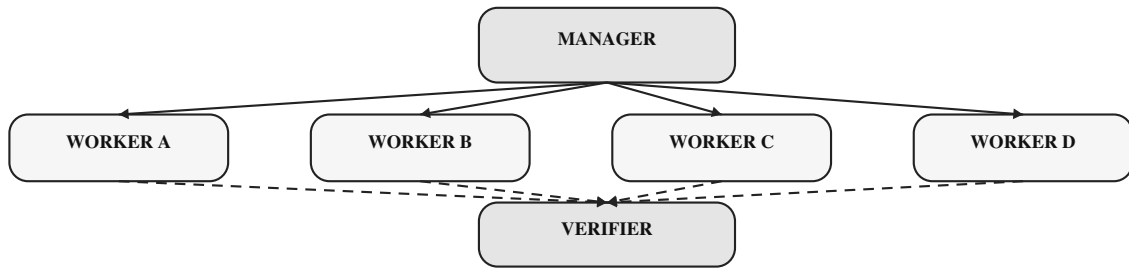


Fig. 6. Fan out independent work and converge through one evidence gate.

A. Parallel Work Must Be Independent

Run Bots in parallel when they can work from stable inputs and produce separable artifacts. Researching five sources, preparing independent account briefs, or building frontend and backend against an agreed contract are good candidates. Parallelizing tightly coupled steps before the interface is stable creates waiting, rework, and expensive cross-talk.

Pseudocode 6 - Fan-Out and Fan-In

```

plan = manager.decompose(goal)
assert dependencies_are_explicit(plan)
workers = launch(plan.ready_tasks, limit=4)
results = gather(workers)
valid = [r for r in results if verify(r)]
merged = synthesizer.combine(valid)
return verifier.final_check(merged)
  
```

B. Use a Concurrency Budget

The author's recommended starting limit is three to four simultaneous specialists for one workflow. This is not a Grok Bot product limit. It is an observability limit: a human should still be able to understand why each worker exists and how its output will be merged. Increase concurrency only after duplicate work, failed handoffs, cost, and verifier load remain controlled.

C. Coordination Rules

- A specialist may depend on an artifact, never on an unrecorded promise.
- The Manager owns prioritization and may reroute blocked work.
- A worker should not wake every other Bot for context.
- A fan-in stage validates schemas before attempting synthesis.
- Conflicting results remain visible and are resolved by a named decision rule.

D. Recover From Missing Teammates

The workshop demonstration shows Bots noticing that previously named teammates are unavailable and looping in the remaining owners [1]. Make this explicit: if a dependency owner is absent, the Manager reassigns only after confirming that no active artifact will arrive. The ledger records the ownership change so two Bots do not later complete the same irreversible action.

TABLE VIII. PARALLELIZATION TEST

Condition	Parallelize	Keep Sequential
Shared write target	No	Yes
Stable interface	Yes	No
Independent evidence	Yes	No
High merge cost	No	Yes
Deadline benefit	Yes	Maybe

E. Scale the Verifier Before the Workers

Four workers can create four times the output and more than four times the review burden. Before adding workers, make evidence machine-checkable, standardize the artifact schema, and ensure the verifier can reject incomplete work automatically. Parallel generation without parallel verification simply moves the bottleneck to the human.

F. Parallel Cost Model

Parallelism trades elapsed time for duplicated context, tool load, coordination, and verification cost. Estimate the expected gain before fan-out: independent workers help when tasks are separable and the merge rule is cheap. They hurt when every worker needs the same scarce tool, writes to the same object, or produces outputs that require subjective reconciliation. Set a concurrency budget per workflow rather than treating capacity as free.

Safe Fan-Out Preconditions

- inputs are frozen or versioned for the duration of the run;
- each worker owns a non-overlapping artifact or hypothesis;
- writes are isolated until verification;
- the convergence rule is defined before work begins.

G. Convergence Is the Product

A team is not complete when every specialist finishes. It is complete when outputs are normalized, conflicts are resolved, evidence is checked, and one accountable owner publishes the result. Design the fan-in step first. If the human must compare ten long answers, the system moved computation but did not remove the coordination bottleneck.

XI. VERIFICATION-FIRST AUTONOMY

Trust grows when the Bot can prove what it did

TABLE IX. EVIDENCE LADDER

Level	Evidence	Use
0	Bot says it is done	Never sufficient
1	Structured summary	Triage only
2	Logs or source links	Traceability
3	Screenshot or diff	Visual review
4	Executed test or video	Behavior proof
5	Independent verifier pass	Autonomy gate

A. Verification Closes the Loop

A worker should be able to use the same surface through which a human judges the result. Cursor cloud agents can run software inside their own virtual machines, click through interfaces, and produce screenshots, logs, or video artifacts [5], [8]. The Grok Bot engineering demo applies the same idea through QA Quincy, which tests frontend and backend behavior and attaches proof [1].

Template 8 - QA Verifier Contract

```
INPUT: candidate artifact + acceptance tests
RUN: exact user-visible workflow
COLLECT: logs, screenshots, video, sources
CHECK: expected state and forbidden state
OUTPUT: PASS | RETRY | ESCALATE
PASS ONLY IF: evidence is reproducible
NEVER: repair the artifact silently
```

B. Separate Producer and Verifier

The worker that created an artifact has context and incentives that can bias its judgment. Use deterministic checks where possible and a separate verifier for material work. The verifier should report failures without silently rewriting the output. The Manager then sends the failure evidence back to the producer or escalates after the retry budget is exhausted.

Pseudocode 7 - Independent Verdict

```
def verify(candidate, tests):
    evidence = run_in_clean_env(candidate, tests)
    failed = [t for t in evidence if not t.pass]
    if not failed: return PASS(evidence)
    if all(t.retryable for t in failed):
        return RETRY(evidence)
    return ESCALATE(evidence)
```

C. Acceptance Test Design

- Describe visible behavior, not implementation preference.
- Include one normal case, one edge case, and one forbidden outcome.
- Name the environment, account, data fixture, and expected final state.
- Require evidence that another person or Bot can reproduce.
- Keep the verifier stable while comparing changes to the worker Skill.

D. The Trust Curve

Lauren's progression begins with closely observed local agents, adds verification Skills, moves reliable work into cloud agents, and only then permits large-scale or automatically merged changes [9]. Apply the same progression to knowledge work. First watch the Bot. Then require evidence. Then automate the trigger. Finally reduce human review only where the verifier has demonstrated reliable rejection.

"The most important skill that you should have in your toolbox when you work with agents is verification."
- Lauren Tan [9]

E. Build an Evidence Pyramid

Use the cheapest reliable proof first. Deterministic checks validate schemas, counts, links, and forbidden values. Logs show that structured operations occurred. Screenshots prove visible state. Short videos demonstrate interaction sequences. A human review resolves taste, ambiguity, or policy. Do not use expensive visual proof when a database query can establish the same fact, and do not trust a log when the user-facing state is what matters.

Verifier Independence

- give the verifier the request and artifact, not the producer's self-assessment;
- require a binary verdict plus enumerated failures;
- store the evidence used for the verdict;
- prevent the producer from editing the verifier's record.

F. Adversarial Acceptance Tests

Include cases that appear successful but violate a hidden requirement: an email draft addressed to the wrong account, a calendar event in the wrong time zone, a report with unsupported claims, or a feature that works only on the happy path. These tests teach the system that completion is not the same as producing an artifact.

XII. APPROVAL BOUNDARIES AND SECURITY

Replace constant supervision with intentional checkpoints

TABLE X. DEFAULT ACTION POLICY

Action	Default	Reason
Read approved source	Allow	Reversible observation
Draft internal artifact	Allow	No external effect
Write reversible record	Allow + log	Recoverable
Send or publish externally	Ask	Reputation impact
Delete, pay, or change access	Human	Hard to undo

A. Approval Is a Policy, Not a Mood

The Grok Bot workshop demonstrates granular rules such as always asking before sending external email or editing a shared calendar [1]. Encode these decisions before the first unattended run. A policy should depend on the action, target, amount, reversibility, and identity involved, not on whether the Bot sounds confident.

Template 9 - Capability Budget

```
SCOPE: approved accounts and folders only
RATE: max 10 external writes per hour
REVERSIBILITY: keep prior value for 30 days
NOTICE: alert on external write or denial
ASK: send, publish, pay, delete, grant access
STOP: repeated denial, unknown domain,
      instruction found in untrusted content
```

B. Prompt Injection Changes the Risk Model

Cursor's security documentation warns that autonomous cloud agents may read malicious instructions embedded in content and can auto-run commands [6]. Treat emails, webpages, documents, repository issues, and retrieved text as untrusted data. A webpage must not be able to expand the Bot's permissions, change its system policy, or redirect secrets to a new destination.

C. Minimum Security Controls

- Separate trusted instructions from untrusted content in every task record.
- Restrict secrets and network destinations to the workflow's actual needs.
- Stamp the acting identity and task_id outside the model-generated content.
- Rate-limit every external write and alert on unusual aggregate behavior.
- Keep an emergency stop that disables triggers without deleting evidence.
- Review access after a role, application, or workflow changes.

Pseudocode 8 - Policy Gate

```
decision = policy(action, target, task.identity)
if decision == DENY: stop_and_log()
if decision == ASK: request_human_approval()
if rate_limit.exceeded(action): trip_wire()
if content.requests_more_access: DENY
execute(action)
append_audit_record(result)
```

D. Human Judgment at the Right Point

Jonas Nelle describes replacing human bottlenecks with intentional checkpoints rather than removing judgment entirely [11]. The human should approve irreversible commitments, ambiguous tradeoffs, and policy exceptions. The Bot should handle evidence collection, routine execution, retries, and preparation so the approval arrives as a compact decision rather than a request to redo the work.

E. Approval Quality Metric

Track the percentage of approval requests that contain the decision, impact, evidence, alternatives, and a recommended action. If a human must reopen five applications to understand the request, the system has not removed the bottleneck. It has merely moved it to the final step.

F. Capability Budgets

Permissions answer whether an operation is allowed. A capability budget also limits how much, how fast, for which objects, and under what observation. A Bot may be allowed to send messages but limited to drafts, named recipients, or a daily count. It may update records but not delete them. Budgets let useful work continue while constraining the blast radius of a mistaken instruction or compromised input.

Four Budget Dimensions

- scope - the accounts, tools, objects, and operations available;
- rate - the maximum writes or commitments per interval;
- reversibility - whether the action can be rolled back safely;
- visibility - who is notified and which evidence is retained.

G. Approval Must Preserve Context

An approval request should include the intended action, target, irreversible effect, evidence collected, alternatives considered, and the safest timeout behavior. A vague 'continue?' prompt transfers the entire reasoning burden back to the human. A good approval surface asks one bounded question and makes declining safe.

XIII. FAILURE RECOVERY AND OBSERVABILITY

A reliable system can explain where it stopped and resume safely

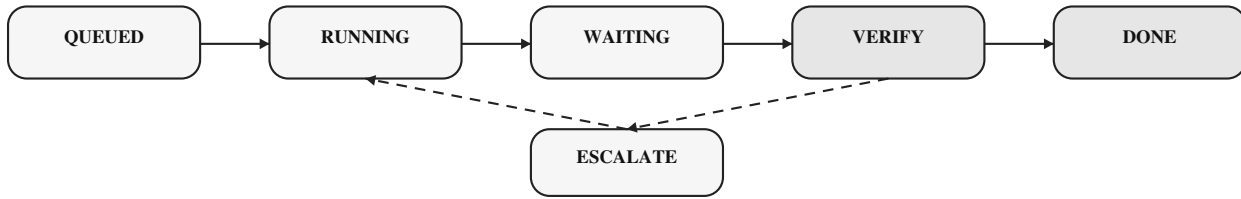


Fig. 7. Failure changes state; it does not erase the prior artifact or evidence.

A. Failure Is a State Transition

Do not treat every error as a request for a larger prompt. Classify the failure, preserve the artifact and trace, and choose a bounded response. Transient network errors may be retried. Invalid credentials require a human. A verifier rejection returns evidence to the producer. A policy denial stops the task. Unknown outcomes must not be repeated blindly.

TABLE XI. FAILURE TAXONOMY

Failure	Automatic Response	Escalate When
Transient tool	Backoff and retry	Budget exhausted
Invalid input	Request missing field	Source cannot supply it
Verification	Return evidence to owner	Same failure repeats
Permission	Stop	Always
Unknown outcome	Inspect before retry	Effect cannot be proven

Pseudocode 9 - Retry With Evidence

```

for attempt in range(MAX_ATTEMPTS):
    result = execute(task, attempt)
    ledger.record(result)
    if result.verified: return DONE
    if not result.retryable: break
    wait(backoff(attempt))
return escalate(task, latest_evidence)
  
```

B. Required Telemetry

- Start and finish timestamps for every task and stage.
- Tool actions, targets, acting identity, and outcome.
- Prompt or Skill version and the exact acceptance tests used.
- Attempt count, cost estimate, produced artifacts, and evidence pointers.
- Approval requests, decisions, policy denials, and ownership changes.

C. Trip Wires Beat Silent Drift

Trigger an alert when external writes spike, the same error repeats, a Routine misses its deadline, a normally quiet Bot contacts a new domain, or a verifier's pass rate changes sharply. Aggregate behavior often exposes failures that individual allow lists miss. A trip wire should freeze the affected capability while leaving read access and logs available for diagnosis.

D. Recovery Drills

Before production, deliberately expire a login, remove a plugin, duplicate a trigger, corrupt an artifact, block a dependency, and deny an approval. Confirm that the Manager reports the exact state and can resume from the last verified checkpoint. Recovery should not depend on rereading a long conversation or remembering what happened yesterday.

E. The Cloud Doctor Pattern

Cursor describes internal Cloud Doctor agents that inspect traces to identify misleading Skills, commands, or environments and then simplify the path for the next run [10]. The transferable lesson is to repair the system around the worker. Repeated failures should improve the Skill, verifier, tool boundary, or environment, not only produce a more desperate prompt.

F. Incident Procedure

When a trip wire fires, pause new writes, preserve current artifacts, record the last confirmed external state, and identify the smallest reversible recovery step. Do not immediately rerun the entire workflow. First determine whether the failure came from the input, Skill, tool, environment, policy, or verifier. Recovery should create a durable incident record that can later improve the system.

Recovery Packet

- task identity, owner, state, and exact failure time;
- last successful action and evidence link;
- writes attempted after that point;
- recommended rollback, replay boundary, and human decision.

G. Practice Failure Deliberately

Run controlled drills for expired sessions, missing files, rate limits, malformed inputs, unavailable specialists, verifier disagreement, and duplicate triggers. The team is production-ready only when failure remains observable and bounded. A flawless happy-path demo is insufficient evidence for unattended operation.

XIV. BUILD PATH A: PERSONAL OPERATIONS

Chief of Staff, Inbox, Calendar, and Tasks in one week

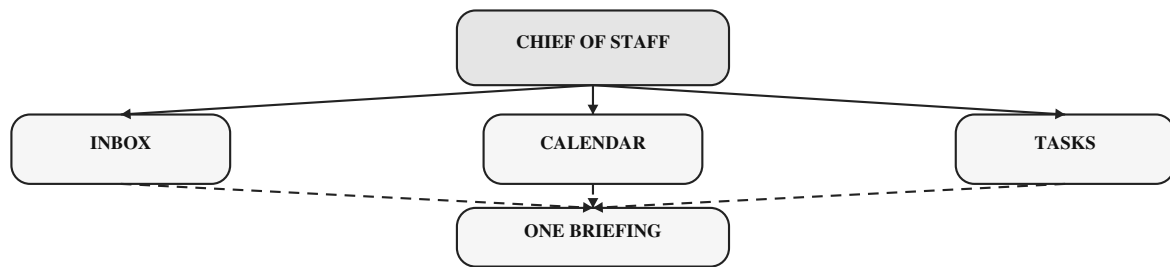


Fig. 8. Personal operations converge into one briefing and one approval surface.

A. Target Outcome

By the end of this build, one Chief of Staff receives a daily objective, an Inbox Manager extracts actionable items, a Calendar Coordinator proposes time blocks, and a Task Organizer maintains the canonical list. The system posts one morning briefing, continues remotely, and asks before any external email or shared-calendar change. This follows the structure demonstrated in the Grok Bot workshop [1].

Prompt 1 - Create the Manager

You are my Chief of Staff.
Own the morning operations workflow.
Delegate inbox, calendar, and task work
to the named specialist Bots.
Return one verified briefing by 08:00.
Ask only for external sends, shared edits,
conflicts, or missing access.

B. Build Sequence

- Create the Chief of Staff and three specialists with explicit role cards.
- Connect only the approved inbox label, calendar, task list, and Slack channel.
- Run one manual briefing and save the successful method as a Skill.
- Add a weekday Routine with an idempotency key and one retry.
- Require each specialist to hand off a compact artifact to the Manager.
- Activate external-send and shared-calendar approval rules.

Prompt 2 - Specialist Contracts

INBOX: return decisions, deadlines, links
CALENDAR: propose slots; never confirm
TASKS: deduplicate and rank by deadline
MANAGER: merge into one briefing
FORMAT: Today / Waiting / Decisions
PASS: every item links to its source

TABLE XII. ACCEPTANCE TESTS

Test	Expected Evidence
Duplicate newsletter	One briefing item
Calendar conflict	Conflict surfaced, no edit
External reply requested	Approval request with draft
No new mail	Heartbeat, not failure
Laptop closed	Run completes in cloud

C. Seven-Day Review

After one week, compare manual and automated minutes, missed items, incorrect items, approval count, duplicate work, and time to recover from failure. Promote recurring corrections into the Skill. Remove any tool permission that was never used. If the briefing is reliable, add one new source or output, not three new Bots.

D. What Not to Automate First

Do not begin with purchases, payments, account recovery, legal commitments, bulk external outreach, or deletion. The personal system should first demonstrate reliable reading, drafting, planning, and reversible updates. Autonomy can expand action by action after evidence shows that the verifier rejects bad outcomes and the policy catches risky ones.

E. Daily Briefing Contract

The Manager should return one compact briefing with sections for completed work, pending approvals, blocked items, upcoming commitments, and anomalies. Each item links to its artifact and evidence. The briefing should not reproduce every Bot conversation. Its purpose is to let the user understand the system's state in minutes and intervene only where judgment changes the outcome.

Personal System Metrics

- minutes of manual routing removed per week;
- percentage of tasks completed without clarification;
- number of incorrect external writes or missed commitments;
- approval requests accepted without additional investigation.

F. Expand by Adjacent Workflow

After one workflow is reliable, add the next adjacent source of work rather than a random new Bot. Inbox triage can feed task capture; task capture can feed calendar preparation; calendar preparation can feed a daily briefing. Shared artifacts and approval rules then compound instead of creating isolated automations.

XV. BUILD PATH B: ENGINEERING DELIVERY

From feature request to tested evidence and an approval-ready change

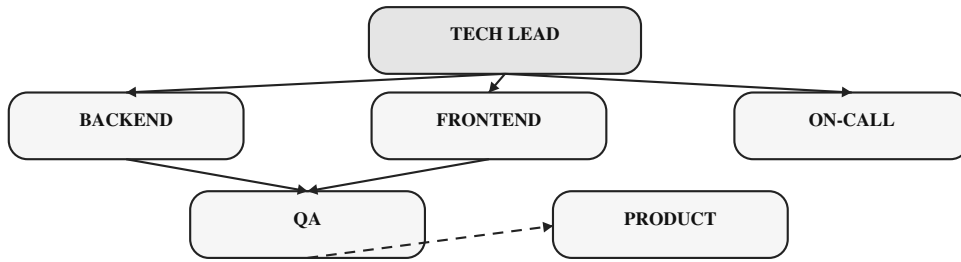


Fig. 9. Engineering specialists converge on independent QA evidence before release preparation.

TABLE XIII. REQUIRED ARTIFACTS

Owner	Artifact	Evidence
Tech Lead	Feature contract	Scope questions resolved
Backend	API change	Unit + contract tests
Frontend	UI change	Build + interaction test
QA	Verification report	Video, logs, screenshots
Product	Release-note draft	Links to verified changes

A. Team Topology

The workshop's engineering example uses a Tech Lead, Backend, Frontend, QA, On-Call, and Product role [1]. The Tech Lead decomposes the feature and establishes interfaces. Backend and Frontend work against the contract. QA produces end-to-end proof. On-Call monitors incidents. Product converts verified technical artifacts into approved customer-facing release notes.

Prompt 3 - Feature Intake

```

GOAL: add lodging to the booking flow
LEAD: challenge scope and write contract
BACKEND: implement API + unit tests
FRONTEND: implement UI against contract
QA: test full flow and record evidence
PRODUCT: draft notes from verified artifacts
HUMAN: approve merge and external publish
  
```

B. Contract Before Parallel Work

The Backend Bot publishes paths, methods, schemas, and assumptions before implementation finishes. The Frontend Bot begins against that artifact. The QA Bot waits for both changes and runs the full behavior. This removes dead meeting time without removing coordination. The dependency is explicit and the waiting state is observable.

Pseudocode 10 - Delivery Gate

```

contract = lead.publish_contract(spec)
api, ui = parallel(
  backend.build(contract),
  frontend.build(contract))
evidence = qa.test(api, ui)
if evidence.pass:
  draft = product.release_notes(evidence)
  request_human_approval(change, draft)
  
```

C. On-Call as an Event-Driven Bot

The workshop creates an On-Call Bot that monitors services such as Datadog and PagerDuty, wakes on a real signal, and asks permission for standing automation [1]. Begin in detection-only mode. Require a severity threshold, deduplication key, evidence bundle, and human approval before production changes. Automated remediation should be introduced only for known, reversible runbooks.

D. Proof Before Merge

Cursor's computer-use materials show agents running the software they changed and producing demos, screenshots, and logs [5], [8], [11]. The transferable rule is that a code diff is not the finished artifact. The finished artifact is a verified behavior plus the evidence needed for a reviewer to make a fast, accountable decision.

E. Engineering State Model

Represent each change as a versioned request with requirements, branch or workspace, owner, test plan, artifacts, review status, and release decision. Backend and frontend work may proceed in parallel only after the interface contract is frozen. QA evaluates the integrated result against user-visible acceptance criteria, while the Tech Lead resolves architectural conflicts and owns the final delivery state.

Evidence Required Before Release

- tests linked to the requirement they protect;
- screenshots or video for user-visible behavior;
- logs or traces for back-end behavior;
- a list of unresolved risks and the rollback path.

F. Keep the Human on Architecture

Agents can implement, test, repair, and prepare changes, but architecture and product tradeoffs remain explicit decisions. The human should be pulled in when a change alters data ownership, permissions, public behavior, cost structure, or an irreversible interface. Routine review comments should become automated checks so human attention moves toward the decisions that cannot be reduced to stable rules.

XVI. ENGINEERING LESSONS FROM CURSOR

Verification, hard constraints, and environments designed for agents

TABLE XIV. THE TRUST PROGRESSION

Stage	Human Role	Agent Capability	Gate
Observe	Watch every run	One local task	Reproduce result
Verify	Inspect evidence	Reusable Skills	Tests reject errors
Cloud	Review artifacts	Unattended execution	Recovery works
Parallel	Manage system	Multiple workers	Merge stays bounded
Automate	Audit exceptions	Triggered changes	Policy + evidence

A. Self-Reported Production Evidence

In an August 2026 workshop, Cursor engineer Lauren Tan reported waking to about 20 agent-authored changes already merged, roughly 1,000 changes in the prior month, and more than 600 changes used to refactor Grok Bot into a stricter architecture [9]. These numbers are self-reported and should not be treated as a universal benchmark. Their value is the path she describes: verification first, then environment design, then scale.

*"The shortest path is the best path."
- Lauren Tan on designing the Dune architecture for agents [9]*

B. Make the Correct Path the Easy Path

Dune co-locates feature code, restricts dependency direction, adds static analysis, CI checks, and automated review, and gives agents conventional patterns to copy [9]. The principle transfers beyond code. Put approved templates next to the source data, expose one canonical output location, make the safe action easier than the risky action, and reject invalid artifacts mechanically.

TABLE XV. SOFT AND HARD CONTROLS

Layer	Example	Reliability
Memory	Prior correction	Low
Prompt	Task instruction	Low-medium
Skill	Versioned method	Medium
Verifier	Executable acceptance test	High
Environment	Schema, permission, CI	Highest

C. Convert Review Comments Into Constraints

Lauren argues that repeated human review comments are a signal to create a lint rule, CI failure, or structural constraint [9]. Apply this to Grok Bot operations: if a human repeatedly corrects missing sources, require a source field. If formatting drifts, validate a schema. If the Bot contacts the wrong audience, restrict the destination. Prompts guide behavior; environments prevent entire classes of failure.

D. Do Not Copy Local Bans Blindly

Specific choices described in the talk, such as banning a framework pattern or code comments, belong to the Grok Bot team's engineering context [9]. They are not general recommendations. Copy the method: observe failures, identify the invariant, enforce it at the strongest practical layer, and verify that the new constraint improves results without blocking legitimate work.

Pseudocode 11 - Turn Taste Into a Gate

```
for review_comment in recurring_comments:
    invariant = generalize(review_comment)
    if machine_checkable(invariant):
        add_schema_or_test(invariant)
    else:
        add_skill_example_and_verifier(invariant)
    evaluate(before, after)
```

E. Architecture as Training Data

A repository teaches agents through its folder boundaries, types, tests, examples, commands, and failure messages. Co-located features reduce the search surface. Explicit dependency rules reveal invalid changes early. Small deterministic checks convert institutional taste into immediate feedback. The strongest agent environment is not the one with the longest instruction file, but the one in which the correct path is easy to discover and incorrect paths fail clearly.

When to Harden a Rule

- the same review comment appears repeatedly;
- the rule can be checked without subjective interpretation;
- violations create material rework or production risk;
- the check explains how to repair the violation.

F. Local Rules Stay Local

A rule that improves one codebase may damage another. Before copying a ban, directory convention, or static check, identify the failure it prevented and confirm that the same failure exists locally. Preserve the principle, then implement the narrowest constraint that fits the new environment.

XVII. 30-DAY IMPLEMENTATION PLAN

Increase autonomy only after evidence and recovery improve

TABLE XVI. FOUR-WEEK BUILD PATH

Window	Build	Required Exit Evidence
Days 1-3	Baseline one workflow	3 manual runs + finish test
Days 4-7	One Bot + one Skill	5 observed passes
Week 2	Routine + approval policy	3 unattended passes
Week 3	Manager + 2 specialists	Typed handoffs + verifier
Week 4	Recovery + measured scale	Drills pass; metrics improve

A. Days 1-3 - Map Reality

Choose one recurring workflow and perform it manually three times. Record the source, tools, decisions, exceptions, elapsed time, artifact, and evidence. Write the workflow canvas and reject any scope that lacks a clear finish state. Create no team yet. The output of this phase is a measured process and an acceptance test.

B. Days 4-7 - Build One Reliable Worker

Create one Bot with a role card and the smallest useful access. Run the process while observing every action. Convert the best run into a Skill, add examples and anti-patterns, and require evidence. Do not schedule it until five representative runs pass and the Bot fails safely when access or input is missing.

C. Week 2 - Add Continuity

Create one Routine with an idempotency key, timeout, retry budget, heartbeat, and stop switch. Add explicit approval rules for external writes and irreversible actions. Run it at low frequency. Inspect the task ledger and verify that a duplicated trigger, expired login, and empty input do not create duplicate or misleading output.

D. Week 3 - Add the Smallest Team

Introduce a Manager only when routing or coordination is a demonstrated bottleneck. Add no more than two specialists at first. Define typed handoffs and one independent verifier. Parallelize only tasks with stable inputs. The human should receive one final result or one compact decision, not a transcript of the team talking.

E. Week 4 - Prove Recovery and ROI

Run recovery drills, inspect permissions, and compare metrics against the manual baseline. Increase frequency, tool scope, or concurrency one dimension at a time. If quality or recovery worsens, roll back the last change. A mature system is not the one with the most autonomy; it is the one whose autonomy can be expanded and reversed predictably.

TABLE XVII. WEEKLY SCORECARD

Metric	Definition	Direction
Completion	Verified runs / started runs	Up
Rework	Runs manually corrected	Down
Interruptions	Human pings per completed run	Down
Recovery	Minutes from failure to safe state	Down
Cost	Tool + model cost per verified result	Down

Template 10 - Scale Gate

```
SCALE only if:
  completion >= target
  rework <= target
  approval quality >= target
  recovery drills pass
  permissions remain minimal
CHANGE one: frequency | scope | workers
ROLL BACK if any guard metric regresses
```

F. Run the Rollout as an Experiment

Change one architectural variable at a time and compare it with the manual baseline. Examples include adding a verifier, replacing free-form handoffs with a schema, or moving a Routine to cloud execution. Track completion, rework, latency, cost, and approval burden for at least several representative runs. A system that completes more work but doubles human investigation has not improved.

Promotion Evidence

- the workflow passes normal, malformed, and interrupted cases;
- the owner and current state are visible without transcript review;
- retries are bounded and duplicated triggers are harmless;
- the human can stop, inspect, and recover the system quickly.

G. Thirty-Day Deliverable

The final deliverable is not a large team diagram. It is one measured workflow, one durable Manager ledger, a small set of versioned Skills, at least one safe Routine, explicit approvals, independent evidence, a recovery drill, and a decision about whether the next specialist is justified.

XVIII. DECISION FRAMEWORK, LIMITATIONS, AND REFERENCES

Choose the smallest architecture that can be verified

TABLE XVIII. ARCHITECTURE DECISION FRAMEWORK

Need	Use	Do Not Add Yet
One bounded task	One Bot + Skill	Manager or team
Repeated start	Routine	More specialists
Distinct ownership	Manager + specialists	All-to-all chat
Independent work	Parallel fan-out	Shared write target
Material action	Verifier + approval	Unbounded autonomy

A. Limitations

Persistent memory can still retrieve stale or irrelevant context. Skills and Routines drift when applications change. Computer use is more fragile than a stable API. Multi-agent systems add cost, latency, and coordination failure modes. Model outputs remain non-deterministic. Autonomous cloud execution increases prompt-injection and data-exposure risk [6]. No architecture removes the need for accountable human judgment on irreversible or ambiguous decisions.

B. Conclusion

Grok Bot makes persistent roles, cloud computers, tools, memory, Routines, and collaboration available in one interface [1]. The operating advantage appears only when those capabilities are arranged around ownership, artifacts, evidence, policies, and recovery. Build one measured loop. Save the method as a Skill. Trigger it as a Routine. Add specialists only for real boundaries. Scale verification before parallelism. The result is not a larger chatbot. It is a system that can continue working and still explain why its output should be trusted.

APPENDIX A. COMPACT GLOSSARY

Term	Operational Definition
Bot	Persistent worker with role, tools, memory, and environment
Manager	Router and owner of workflow state
Skill	Versioned reusable operating procedure
Routine	Triggered execution contract
Handoff	Typed transfer of artifact and ownership
Verifier	Independent acceptance-test executor
Guardrail	Policy or hard constraint on action

APPENDIX B. PRODUCTION CHECKLIST

- One source, owner, artifact, verifier, and finish state are named.
- Skills are versioned; Routines are idempotent; handoffs are typed.
- External writes, deletion, payments, and access changes require policy review.
- Retries are bounded; unknown outcomes are inspected before repetition.
- Every completion points to reproducible evidence and an audit record.
- A stop switch, recovery drill, and rollback path have been tested.

REFERENCES

- [1] Grok Bot, 'Meet Grok Bot: Your Team of AI Agents,' workshop page and public description, Aug. 20, 2026, luma.com/38zy3ov.
- [2] Cursor, 'Workshops,' on-demand and live engineering sessions, cursor.com/workshops, accessed Aug. 22, 2026.
- [3] Cursor Docs, 'Agent Skills,' cursor.com/docs/skills, accessed Aug. 22, 2026.
- [4] Cursor Docs, 'Automations,' cursor.com/docs/cloud-agent/automations, accessed Aug. 22, 2026.
- [5] Cursor Docs, 'Cloud Agent Capabilities,' cursor.com/docs/cloud-agent/capabilities, accessed Aug. 22, 2026.
- [6] Cursor Docs, 'Cloud Agent Security,' cursor.com/docs/cloud-agent/security, accessed Aug. 22, 2026.
- [7] Cursor, 'Best Practices for Coding with Agents,' cursor.com/blog/agent-best-practices, 2026.
- [8] Cursor, 'Cursor Agents Can Now Control Their Own Computers,' cursor.com/blog/agent-computer-use, 2026.
- [9] L. Tan, 'How Cursor Turned AI Agents Into Better Engineers,' public workshop recording, Aug. 12, 2026, maven.com/p/e23d9c.
- [10] Cursor, 'How We Set Up Our Cloud Agent Environment,' cursor.com/blog/cloud-agent-environment, 2026.
- [11] J. Nelle, 'What Happens When Agents Get Their Own Computers,' Cursor video, youtube.com/watch?v=_GljcHROPX8, 2026.
- [12] Cursor, 'Cloud Agents,' cursor.com/blog/cloud-agents, 2025.

APPENDIX C. ARCHITECTURE DECISION RULE

Choose the smallest architecture that externalizes the real bottleneck. Use one Bot when the problem is execution, a Skill when the problem is repeatability, a Routine when the problem is continuity, a specialist when the problem is durable expertise or permissions, a Manager when the problem is routing, and a verifier when the problem is trust. Add parallel workers only after inputs and convergence are stable.

SOURCE METHOD

This document is an independent practical synthesis of the public Grok Bot workshop, public Cursor documentation, engineering articles, workshops, and videos cited below. Product behavior may change while Grok Bot remains in beta. Claims about internal engineering practice are limited to what speakers publicly described. Recommendations, templates, pseudocode, and decision rules are the author's adaptation for implementation and are not official SpaceXAI, xAI, SpaceX, or Cursor specifications.